COMPUTER SCIENCE & GRAPH THEORY

Dijkstra's Shortest Path Algorithm

A Complete Masterclass on Single-Source Shortest Paths, Greedy Relaxation, Heap Optimization & Real-World Applications

DIJKSTRA'S SHORTEST PATH ALGORITHM

Core Foundations: Single-Source Shortest Paths on Weighted Graphs

01

Single-Source Scope

Computes the exact minimum distance from a fixed source vertex to every other accessible vertex in a weighted graph.

Outputs a complete shortest path tree originating from the source node.

02

Greedy Edge Relaxation

Repeatedly selects the unvisited vertex with the smallest tentative distance and relaxes its outgoing edges.

Relaxation Rule:
dist[v] = min(dist[v], dist[u] + weight(u, v))

03

Non-Negative Constraint

Requires all edge weights to be strictly non-negative (weight >= 0) to guarantee greedy choice optimality.

Negative edge weights violate the greedy assumption, requiring Bellman-Ford instead.

DIJKSTRA'S SHORTEST PATH ALGORITHM

Algorithmic Mechanics: Step-by-Step State Traversal

STEP 1

Initialization

• Set dist[source] = 0
• Set dist[v] = ∞ for all other vertices v
• Place all vertices into unvisited priority queue Q.

STEP 2

Node Selection

• Extract vertex u from Q with the minimum tentative distance dist[u].
• Mark u as visited (it is never re-processed).

STEP 3

Edge Relaxation

• For each unvisited neighbor v of u:
  Calculate alt = dist[u] + weight(u, v).
• If alt < dist[v], update dist[v] = alt and set prev[v] = u.

STEP 4

Termination

• Repeat Steps 2–3 until Q becomes empty.
• Result: Guaranteed minimum distance table and shortest path tree.

Complexity Analysis: Data Structure Trade-offs

Array / Adjacency Matrix

Time Complexity: $O(V^2)$

Space Complexity: $O(V^2)$

Vertex Lookup: O(V) linear search

Optimal For: Dense graphs where $E \approx V^2$

Key Limitation: Inflexible and inefficient for sparse networks (e.g. road maps)

Min-Heap / Adjacency List

Time Complexity: $O((V + E) \log V)$

Space Complexity: $O(V + E)$

Vertex Lookup: O(log V) priority queue pop

Optimal For: Sparse graphs where $E \ll V^2$

Industry Standard: Powers large-scale routing in GPS and network protocols

DIJKSTRA'S SHORTEST PATH ALGORITHM

Python Implementation: Min-Heap & Edge Relaxation

```
import heapq

def dijkstra(graph, start):
distances = {node: float('inf') for node in graph}
distances[start] = 0
pq = [(0, start)]  # (distance, node)

while pq:
curr_dist, curr_node = heapq.heappop(pq)

if curr_dist > distances[curr_node]:
continue

for neighbor, weight in graph[curr_node].items():
distance = curr_dist + weight
if distance < distances[neighbor]:
distances[neighbor] = distance
heapq.heappush(pq, (distance, neighbor))

return distances
```

Implementation Highlights

- **Priority Queue via heapq:** Stores tuples of (distance, node) to ensure logarithmic extraction of the minimum distance vertex.
- **Stale State Pruning:** The check `curr_dist > distances[curr_node]` skips outdated heap entries without needing explicit decrease-key operations.
- **Graph Representation:** Represented as an adjacency list dictionary where `graph[node]` contains neighbor weights.

DIJKSTRA'S SHORTEST PATH ALGORITHM

Real-World Impact & Industry Applications

GPS & Digital Mapping

Calculates optimal driving routes and minimum travel time across road networks in software like Google Maps.

IP Network Routing

Powers link-state routing protocols such as Open Shortest Path First (OSPF) to route internet packets with minimal latency.

Robotics & Motion Planning

Determines collision-free, minimal-distance trajectories for autonomous mobile robots and manufacturing systems.

Game AI Pathfinding

Enables non-player characters (NPCs) to dynamically navigate complex game maps and obstacle environments.